

SQL CHEATSHEET

BY- SUKHVINDER SINGH

CREATE DATABASE:

`CREATE DATABASE databasename;`

USE COMMAND:

The USE statement of MySQL helps you to select/use a database. You can also change to another database with this statement. Once you set the current database it will be same until the end of the session unless you change the it.

`USE Person;`

CREATE TABLES:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
);
```

Eg:

```
CREATE TABLE Persons (  
    PersonID int,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address varchar(255),  
    City varchar(255)  
);
```

In MySQL there are three main data types: string, numeric, and date and time.

String Data Types:

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large Objects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LOBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Numeric Data Types:

Data type	Description
BIT(size)	A bit-value type. The number of bits per value is specified in <i>size</i> . The <i>size</i> parameter can hold a value from 1 to 64. The default value for <i>size</i> is 1.
TINYINT(size)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The <i>size</i> parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(size)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The <i>size</i> parameter specifies the maximum display width (which is 255)
MEDIUMINT(size)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The <i>size</i> parameter specifies the maximum display width (which is 255)
INT(size)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The <i>size</i> parameter specifies the maximum display width (which is 255)
INTEGER(size)	Equal to INT(size)
BIGINT(size)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The <i>size</i> parameter specifies the maximum display width (which is 255)

FLOAT(<i>size</i> , <i>d</i>)	A floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(<i>p</i>)	A floating point number. MySQL uses the <i>p</i> value to determine whether to use FLOAT or DOUBLE for the resulting data type. If <i>p</i> is from 0 to 24, the data type becomes FLOAT(). If <i>p</i> is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(<i>size</i> , <i>d</i>)	A normal-size floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter
DOUBLE PRECISION(<i>size</i> , <i>d</i>)	
DECIMAL(<i>size</i> , <i>d</i>)	An exact fixed-point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. The maximum number for <i>size</i> is 65. The maximum number for <i>d</i> is 30. The default value for <i>size</i> is 10. The default value for <i>d</i> is 0.
DEC(<i>size</i> , <i>d</i>)	Equal to DECIMAL(<i>size</i> , <i>d</i>)

Date and Time Data Types:

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(<i>fsp</i>)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(<i>fsp</i>)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(<i>fsp</i>)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

INSERT INTO Statement:

To insert data into a MySQL table, you would need to use the SQL **INSERT INTO** command.

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Eg:

```
INSERT INTO Persons (PersonID, LastName, FirstName, Address, City)
VALUES (1, 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```

INSERT MULTIPLE ROWS:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES
(value1, value2, value3, ...),
(value1, value2, value3, ...),
(value1, value2, value3, ...),
(value1, value2, value3, ...);
```

Eg:

```
INSERT INTO Persons (PersonID, LastName, FirstName, Address)
VALUES
(3, 'Tom A. Erichsen', 'Skagen', '44660' ),
(4, 'Tom B. Erichsen', 'Skagen', '44660' ),
(5, 'Tom C. Erichsen', 'Skagen', '44660' ),
(6, 'Tom D. Erichsen', 'Skagen', '44660' );
```

SQL Constraints

SQL constraints are used to specify rules for data in a table.

- [NOT NULL](#) - Ensures that a column cannot have a NULL value
- [UNIQUE](#) - Ensures that all values in a column are different
- [PRIMARY KEY](#) - A combination of a **NOT NULL** and **UNIQUE**. Uniquely identifies each row in a table
- [FOREIGN KEY](#) - Prevents actions that would destroy links between tables
- [CHECK](#) - Ensures that the values in a column satisfies a specific condition
- [DEFAULT](#) - Sets a default value for a column if no value is specified
- [CREATE INDEX](#) - Used to create and retrieve data from the database very quickly

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    ....  
);
```

```
CREATE TABLE Persons (  
    ID int NOT NULL UNIQUE,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int CHECK (Age>=18),  
    City varchar(255) DEFAULT 'Delhi'  
);
```

```
INSERT INTO Persons (PersonID, Name, Age, Gender, Phone)  
VALUES
```

```
(3, 'Tom A.', 18, 'Male', '8794563652' ),  
(4, 'Tom B.', 19, 'Female', '8569854785' ),  
(5, 'Tom C.', 18, 'Female', '9658745698' ),  
(6, 'Tom D.', 25, 'Male', '7854698563' );
```

The SQL SELECT Statement

```
SELECT column1, column2, ...  
FROM table_name;
```

Here, column1, column2, ... are the field names of the table you want to select data from. If you want to select all the fields available in the table, use the following syntax:

```
SELECT * FROM table_name;
```

Note: In general, it is not good practice to use an asterisk (*) in the select statement if you don't really need all the columns. It will automatically query everything, which increases traffic between the database server and the application and can slow the retrieval of results.

Example:

```
SELECT PersonID, Name, Age  
FROM Persons;
```

SQL Aliases

- SQL aliases are used to give a table, or a column in a table, a temporary name.
- Aliases are often used to make column names more readable.
- An alias only exists for the duration of that query.
- An alias is created with the **AS** keyword.

```
SELECT column_name AS alias_name  
FROM table_name;
```

Eg:

```
SELECT PersonID AS Id ,Name AS Person_Name, Age AS Person_Age  
FROM Persons;
```

```
SELECT PersonID AS Id, Name AS "Person Name", Age AS "Person Age"  
FROM Persons;
```

Alias Table Syntax

```
SELECT column_name(s)  
FROM table_name AS alias_name;
```

The SQL WHERE Clause

The **WHERE** clause is used to filter records.

It is used to extract only those records that fulfill a specified condition.

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Eg:

```
SELECT * FROM Persons  
WHERE Age>18;
```

```
SELECT * FROM Persons  
WHERE Age>=19;
```

```
SELECT * FROM Persons  
WHERE Gender="Male";
```

Operators in The WHERE Clause:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

The SQL AND, OR and NOT Operators:

The **WHERE** clause can be combined with **AND**, **OR**, and **NOT** operators.

The **AND** and **OR** operators are used to filter records based on more than one condition:

- The **AND** operator displays a record if all the conditions separated by **AND** are TRUE.
- The **OR** operator displays a record if any of the conditions separated by **OR** is TRUE.

The **NOT** operator displays a record if the condition(s) is NOT TRUE.

AND Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

Eg:

```
SELECT * FROM Persons  
WHERE Age>=19 AND Age<25;
```

```
SELECT * FROM Persons  
WHERE Age>=19 AND Gender="Male";
```

OR Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

Eg:

```
SELECT * FROM Persons  
WHERE Age>=19 OR Age<25;
```

```
SELECT * FROM Persons  
WHERE (Age>=19 OR Age<25) AND Gender="Male";
```


NOT Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```

Eg:

```
SELECT * FROM Persons  
WHERE NOT Gender="Male";
```

```
SELECT * FROM Persons  
WHERE NOT (Age>20 OR Gender="Male");
```

The SQL IN Operator:

- The **IN** operator allows you to specify multiple values in a **WHERE** clause.
- The **IN** operator returns table tuple for which the condition satisfy.

IN Syntax

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name IN (value1, value2, ...);
```

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name NOT IN (value1, value2, ...);
```

Eg:

```
SELECT * FROM Persons  
WHERE Age IN (18,21);
```

```
SELECT * FROM Persons
WHERE Age NOT IN (18,21);
```

The SQL BETWEEN Operator:

The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.

The **BETWEEN** operator is inclusive: begin and end values are included.

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Eg:

```
SELECT * FROM Persons
WHERE Age BETWEEN 18 AND 20;
```

```
SELECT * FROM Persons
WHERE DOB BETWEEN "1998-01-01" AND "1998-06-30";
```

```
SELECT * FROM Persons
WHERE Age NOT BETWEEN 18 AND 20;
```

```
SELECT * FROM Persons
WHERE Name BETWEEN "a" AND "k";
```

```
SELECT * FROM Persons
WHERE Name BETWEEN "anil" AND "kamal";
```

******There are two names mention but it will search only first character i.e. "a" and "k"

The SQL LIKE Operator:

There are two wildcards often used in conjunction with the **LIKE** operator:

- The percent sign (%) represents zero, one, or multiple characters
- The underscore sign (_) represents one, single character

LIKE Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE columnN LIKE pattern;
```

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Eg:

The SQL statement selects all customers with a CustomerName starting with "a":

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';
```

The SQL statement selects all customers with a CustomerName Not starting with "a":

```
SELECT * FROM Customers
WHERE CustomerName NOT LIKE 'a%';
```

The SQL statement selects all customers with a ContactName that starts with "a" and ends with "k":

```
SELECT * FROM Customers
WHERE ContactName LIKE 'a%k';
```

SQL – Regexp:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE columnN REGEXP pattern;
```

Student Table

Name	Age	Gender
Ram Kumar	19	Male
Salman Khan	18	Male
Meera Khan	19	Female
Sarita Kumari	21	Female
Anil Kapoor	20	Male

WHERE Name REGEXP "khan\$ | poor"



Name	Age	Gender
Salman Khan	18	Male
Meera Khan	19	Female
Anil Kapoor	20	Male

Pattern	What the pattern matches
^	Beginning of string
\$	End of string
.	Any single character
[...]	Any character listed between the square brackets
[^...]	Any character not listed between the square brackets
p1 p2 p3	Alternation; matches any of the patterns p1, p2, or p3
*	Zero or more instances of preceding element
+	One or more instances of preceding element
{n}	n instances of preceding element
{m,n}	m through n instances of preceding element

Eg:

```
SELECT * FROM Persons  
WHERE Name REGEXP '^w';
```

```
SELECT * FROM Persons  
WHERE Gender REGEXP 'fe';
```

The SQL statement selects all Person with a Name that starts with "s":

```
SELECT * FROM Persons  
WHERE Name REGEXP '^s';
```

The SQL statement selects all Person with a Name that Ends with "s":

```
SELECT * FROM Persons  
WHERE Name REGEXP 's$';
```

The SQL statement selects all Person with a Name that includes char "s" and "m":

```
SELECT * FROM Persons  
WHERE Name REGEXP '[sm]';
```

The SQL statement selects all Person with a Name that includes of "ra" and "ma":

```
SELECT * FROM Persons  
WHERE Name REGEXP '[rm]a';
```

//combination

// ra

// ma

The SQL statement selects all Person with a Name that includes of "ra" ,"ma" and "ha":

```

SELECT * FROM Persons
WHERE Name REGEXP '[rmh]a';
//combination
// ra
// ma
// ha

```

The SQL statement selects all Person with a Name that includes of "sr" ,"sm" and "sh":

```

SELECT * FROM Persons
WHERE Name REGEXP 's[rmh]';
//combination
// sr
// sm
// sh

```

The SQL statement selects all Person with a Name that starts with "s" ,"f" and "z":

```

SELECT * FROM Persons
WHERE Name REGEXP '^[sfz]';

```

The SQL statement selects all Person with a Name that Ends with "s" ,"f" and "z":

```

SELECT * FROM Persons
WHERE Name REGEXP '[sfz]$';

```

The SQL statement selects all Person with a Name that Includes "a" to "k" range char:

```

SELECT * FROM Persons
WHERE Name REGEXP '[a-k]';

```

The SQL statement selects all Person with a Name that Includes "sa" ,"sb" ,"sc" ,"sd" ,"se"
 ,"sk" range char:

```
SELECT * FROM Persons
```

```
WHERE Name REGEXP 's[a-k]';
```

The SQL statement selects all Person with a Name that Includes "as", "bs" , "cs", "ds" , "es" , "ks" range char:

```
SELECT * FROM Persons
```

```
WHERE Name REGEXP '[a-k]s';
```

The SQL ORDER BY Keyword

The **ORDER BY** keyword is used to sort the result-set in ascending or descending order.

The **ORDER BY** keyword sorts the records in ascending order by default. To sort the records in descending order, use the **DESC** keyword.

ORDER BY Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

Eg:

```
SELECT * FROM Persons  
ORDER BY Name ;
```

```
SELECT * FROM Persons  
ORDER BY Name DESC;
```

```
SELECT * FROM Persons  
ORDER BY Name ASC;
```

```
SELECT * FROM Persons  
WHERE Age >= 20  
ORDER BY Name ASC;
```

ORDER BY Several Columns Example:

The following SQL statement selects all customers from the "Customers" table, sorted by the "Country" and the "CustomerName" column. This means that it orders by Country, but if some rows have the same Country, it orders them by CustomerName:

```
SELECT * FROM Customers
ORDER BY Country, CustomerName;
```

Sorting with NULLs

By default:

- **NULLS** appear **last** in ascending order.
- **NULLS** appear **first** in descending order.

You can override this behaviour:

```
SELECT * FROM Customers
ORDER BY Country ASC NULLS FIRST;
```

Note:

- Use ORDER BY **at the end** of your SQL query.
- Sorting can be performance-intensive for large datasets — use indexes if needed.

The SQL SELECT DISTINCT Statement

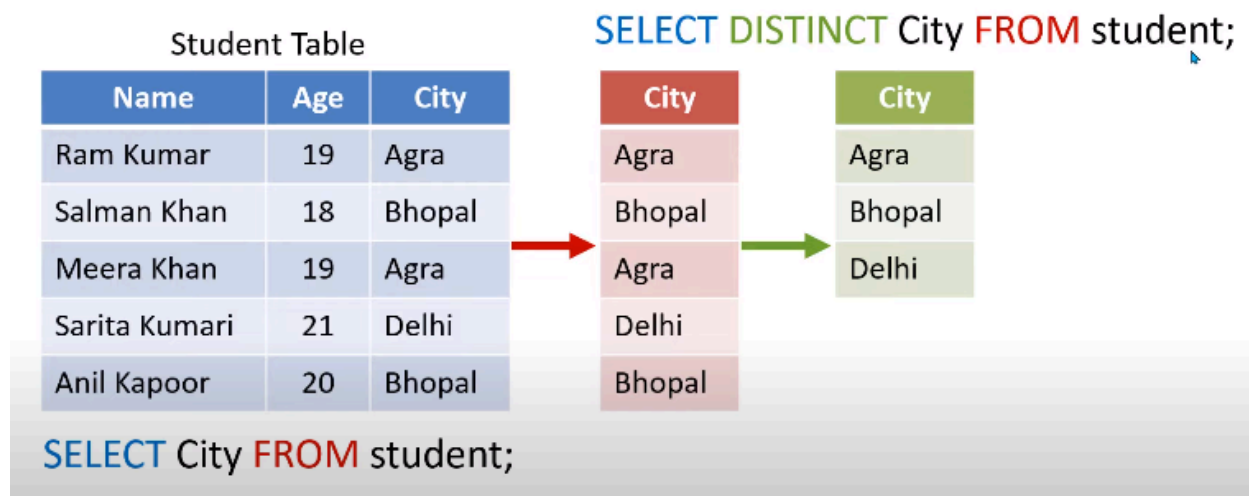
The **SELECT DISTINCT** statement is used to return only distinct (different) values.

Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.

SELECT DISTINCT Syntax:

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```


SELECT Data with DISTINCT



Example:

```
SELECT DISTINCT City  
FROM Persons;
```

```
SELECT DISTINCT City  
FROM Persons;  
ORDER BY Age;
```

Multiple Columns

```
SELECT DISTINCT department, salary  
FROM employees;
```

Purpose: Returns unique combinations of **department** and **salary**. If two employees are in the same department with the same salary, only one row is returned for that combination.

⚠ Notes:

- **DISTINCT** affects **all selected columns**, not just the first one.
- Use it **only when needed**, as it may slow down queries for large datasets

SQL NULL Values:

A NULL value is different from a zero value or a field that contains spaces. A field with a NULL value is one that has been left blank during record creation!

How to Test for NULL Values?

It is not possible to test for NULL values with comparison operators, such as =, <, or <>.

We will have to use the **IS NULL** and **IS NOT NULL** operators instead.

IS NULL Syntax

```
SELECT column_names  
FROM table_name  
WHERE column_name IS NULL;
```

IS NOT NULL Syntax

```
SELECT column_names  
FROM table_name  
WHERE column_name IS NOT NULL;
```

SQL LIMIT & OFFSET:

The SQL statement selects starting 3 Person:

```
SELECT * FROM Persons  
LIMIT 3;
```



The SQL Offset statement skips mentioned value, here it skip starting 3 value and show 4 vlaue as in limit:

```
SELECT * FROM Persons  
LIMIT 3,4;
```

The SQL COUNT(), AVG(), MIN(), MAX() and SUM() Functions

The **COUNT()** function returns the number of rows that matches a specified criterion.

COUNT(*) is often faster than COUNT(column) since it doesn't check for NULL values.

COUNT() Syntax

```
SELECT COUNT(column_name)  
FROM table_name  
WHERE condition;
```

Eg:

```
SELECT COUNT (Name) As NameCount  
FROM Persons;  
SELECT COUNT (DISTINCT City) As DistictCityCount  
FROM Persons;
```

The **MAX()** function returns the largest value of the selected column.

MAX() Syntax

```
SELECT MAX(column_name)  
FROM table_name  
WHERE condition;
```

Eg:

```
SELECT MAX(Age) As DistictCityCount  
FROM Persons;
```

The **MIN()** function returns the smallest value of the selected column.

MIN() Syntax

```
SELECT MIN(column_name)  
FROM table_name  
WHERE condition;
```

Eg:

```
SELECT MIN(Age) As DistictCityCount  
FROM Persons;
```

The **SUM()** function returns the total sum of a numeric column.

SUM() Syntax

```
SELECT SUM(column_name)  
FROM table_name  
WHERE condition;
```

The **AVG()** function returns the average value of a numeric column.

AVG() Syntax

```
SELECT AVG(column_name)  
FROM table_name  
WHERE condition;
```

The SQL UPDATE Statement:

The **UPDATE** statement is used to modify the existing records in a table.

Note: Be careful when updating records in a table! Notice the **WHERE** clause in the **UPDATE** statement. The **WHERE** clause specifies which record(s) that should be updated. If you omit the **WHERE** clause, all records in the table will be updated!

UPDATE Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

```
UPDATE persons  
SET Name = 'Sukhvinder Singh', City= 'Meerut'  
WHERE PersonID = 3;
```

```
UPDATE Persons  
SET Age=21  
WHERE PersonID IN (3,4,5);
```

COMMIT and ROLLBACK in SQL

What is SQL COMMIT?

A COMMIT is the SQL command used **in the transaction tables or database** to make the current transaction or database **statement** as permanent. we want to make the changes permanent. We need to perform the **commit command** to save the changes, and these changes become permanent for all users. Furthermore, once the commit command is executed in the database, we cannot regain its previous states in which it was earlier before the execution of the first statement.

Syntax

```
COMMIT;
```

Eg:

```
DELETE from Employees Where age = 27;
```

COMMIT;

In the above example, we will delete all those records from the **EMPLOYEES** table whose age is 27 and then **COMMIT** query to make the changes as permanent that visible for all users in the database records.

What is SQL ROLLBACK?

A rollback command performs the current transaction's rollback action to return the transaction on its **previous state** or the first statement.

A rollback command can only be executed if the user has not performed the **COMMIT** command on the current transaction or statement.

Syntax

ROLLBACK;

Eg:

DELETE from Employees **Where** age = **27**;
ROLLBACK;

ROLLBACK AND COMMIT WORKS FOR:

- **INSERT**
- **UPDATE**
- **DELETE**

The SQL DELETE Statement:

The **DELETE** statement is used to delete existing records in a table.

DELETE Syntax:

DELETE FROM *table_name* **WHERE** *condition*;

Note: Be careful when deleting records in a table! Notice the **WHERE** clause in the **DELETE** statement. The **WHERE** clause specifies which record(s) should be deleted. If you omit the **WHERE** clause, all records in the table will be deleted!

SQL DELETE Example:

The following SQL statement deletes the customer "Alfreds Futterkiste" from the "Customers" table:

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

Delete All Records:

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:

```
DELETE FROM table_name;
```

SQL ALTER TABLE Statement

The **ALTER TABLE** statement is used to add, delete, or modify columns in an existing table.

The **ALTER TABLE** statement is also used to add and drop various constraints on an existing table.

ALTER TABLE - ADD Column

To add a column in a table, use the following syntax:

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Example

```
ALTER TABLE Customers  
ADD Email varchar(255);
```

ALTER TABLE - DROP COLUMN

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example

```
ALTER TABLE Customers  
DROP COLUMN Email;
```

ALTER TABLE - ALTER/MODIFY COLUMN

To change the data type of a column in a table, use the following syntax:

SQL Server / MS Access:

```
ALTER TABLE table_name  
ALTER COLUMN column_name datatype;
```

My SQL :

```
ALTER TABLE table_name  
MODIFY COLUMN column_name datatype;
```

RENAME COLUMN

To change the name of a column in a table, use the following syntax:

SQL:

```
ALTER TABLE table_name  
CHANGE column_name new_name datatype;
```

Eg:

```
ALTER TABLE table_name  
CHANGE Name FirstName Varchar(50);
```


SQL ALTER TABLE Example:

```
ALTER TABLE Persons  
ADD DateOfBirth date;
```

```
ALTER TABLE Persons  
ADD PRIMARY KEY (Id);
```

Change Data Type Example:

```
ALTER TABLE Persons  
ALTER COLUMN DateOfBirth year;
```

SQL PRIMARY KEY Constraint:

The **PRIMARY KEY** constraint uniquely identifies each record in a table.

Primary keys must contain UNIQUE values, and cannot contain NULL values.

A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

SQL PRIMARY KEY on CREATE TABLE:

The following SQL creates a **PRIMARY KEY** on the "ID" column when the "Persons" table is created:

Syntax

```
CREATE TABLE Persons (  
    ID int NOT NULL PRIMARY KEY AUTO_INCREMENT,  
    LastName varchar(255) NOT NULL,
```

```
    FirstName varchar(255),  
    Age int  
);  
OR
```

```
CREATE TABLE Persons (  
    ID int NOT NULL AUTO_INCREMENT,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    PRIMARY KEY (ID)  
);
```

On Already created table:

```
ALTER TABLE Persons  
ADD PRIMARY KEY (Id);
```

SQL FOREIGN KEY Constraint:

The **FOREIGN KEY** is a key used to link two tables.

The **FOREIGN KEY** constraint is used to prevent actions that would destroy links between tables.

A **FOREIGN KEY** is a field (or collection of fields) in one table, that refers to the [PRIMARY KEY](#) in another table.

The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

Persons Table

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

Orders Table

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Notice that the "PersonID" column in the "Orders" table points to the "PersonID" column in the "Persons" table.

The "PersonID" column in the "Persons" table is the **PRIMARY KEY** in the "Persons" table.

The "PersonID" column in the "Orders" table is a **FOREIGN KEY** in the "Orders" table.

The **FOREIGN KEY** constraint prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the parent table.

SQL FOREIGN KEY on CREATE TABLE

The following SQL creates a **FOREIGN KEY** on the "PersonID" column when the "Orders" table is created:

MySQL:

```
CREATE TABLE Orders (  
    OrderID int NOT NULL,  
    OrderNumber int NOT NULL,  
    PersonID int,  
    PRIMARY KEY (OrderID),  
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
);
```

SQL Server :

```
CREATE TABLE Orders (  
    OrderID int NOT NULL PRIMARY KEY,  
    OrderNumber int NOT NULL,  
    PersonID int FOREIGN KEY REFERENCES Persons(PersonID)  
);
```

SQL FOREIGN KEY on ALTER TABLE:

To create a **FOREIGN KEY** constraint on the "PersonID" column when the "Orders" table is already created, use the following SQL:

MySQL / SQL Server / Oracle / MS Access:

ALTER TABLE Orders

ADD FOREIGN KEY (PersonID) **REFERENCES** Persons(PersonID);

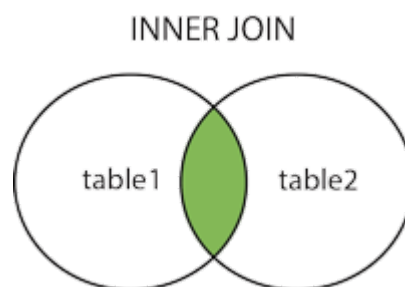
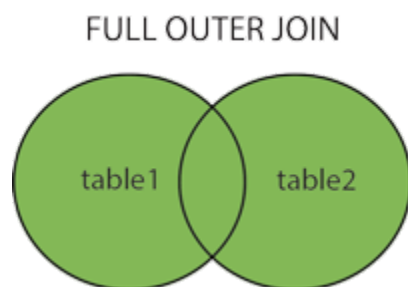
SQL JOIN:

A **JOIN** clause is used to combine rows from two or more tables, based on a related column between them.

Different Types of SQL JOINS

Here are the different types of the JOINS in SQL:

- **(INNER) JOIN**: Returns records that have matching values in both tables
- **LEFT (OUTER) JOIN**: Returns all records from the left table, and the matched records from the right table
- **RIGHT (OUTER) JOIN**: Returns all records from the right table, and the matched records from the left table
- **CROSS JOIN OR FULL JOIN** : The **CROSS JOIN** keyword returns all records from both tables (table1 and table2).





SQL INNER JOIN Keyword:

The **INNER JOIN** keyword selects records that have matching values in both tables. They are also referred to as an **EQUIJOIN**

The INNER JOIN creates a new result table by combining column values of two tables (table1 and table2) based upon the join-predicate.

INNER JOIN Syntax

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Ussing Alias:

```
SELECT column_name(s)
FROM table1 t1
INNER JOIN table2 t2
ON t1.column_name = t2.column_name;
```

Example

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers
ON Orders.CustomerID = Customers.CustomerID;
```

Table 1 – CUSTOMERS Table is as follows.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Table 2 – ORDERS Table is as follows.

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

This would produce the following result.

ID	NAME	AMOUNT	DATE
3	kaushik	3000	2009-10-08 00:00:00
3	kaushik	1500	2009-10-08 00:00:00
2	Khilan	1560	2009-11-20 00:00:00
4	Chaitali	2060	2008-05-20 00:00:00

Using WHERE or ORDER BY clause:

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID
WHERE Customers.city = "Delhi"
ORDER BY Customers.CustomerName;
```

SQL LEFT JOIN Keyword

The **LEFT JOIN** keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

The SQL **LEFT JOIN** returns all rows from the left table, even if there are no matches in the right table. This means that if the ON clause matches 0 (zero) records in the right table; the join will still return a row in the result, but with NULL in each column from the right table.

LEFT JOIN Syntax

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Example

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

SQL RIGHT JOIN Keyword:

The SQL **RIGHT JOIN** returns all rows from the right table, even if there are no matches in the left table. This means that if the ON clause matches 0 (zero) records in the left table; the join will still return a row in the result, but with NULL in each column from the left table.

This means that a right join returns all the values from the right table, plus matched values from the left table or NULL in case of no matching join predicate.

RIGHT JOIN Syntax:

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

SQL CROSS JOIN OR CARTESIAN JOIN

Keyword:

The CARTESIAN JOIN or CROSS JOIN returns the Cartesian product of the sets of records from two or more joined tables. Thus, it equates to an inner join where the join-condition always evaluates to either True or where the join-condition is absent from the statement.

The **CROSS JOIN** keyword returns all records from both tables (table1 and table2).

CROSS JOIN Syntax

```
SELECT column_name(s)
FROM table1
CROSS JOIN table2;
```

MySQL CROSS JOIN Example:

The following SQL statement selects all customers, and all orders:

Example

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
CROSS JOIN Orders;
```

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers , Orders;
```

Example

Consider the following two tables.

Table 1 – CUSTOMERS table is as follows.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Table 2: ORDERS Table is as follows –

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

Now, let us join these two tables using CARTESIAN JOIN as follows –

```
SQL> SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS, ORDERS;
```

This would produce the following result –

ID	NAME	AMOUNT	DATE
1	Ramesh	3000	2009-10-08 00:00:00
1	Ramesh	1500	2009-10-08 00:00:00
1	Ramesh	1560	2009-11-20 00:00:00
1	Ramesh	2060	2008-05-20 00:00:00
2	Khilan	3000	2009-10-08 00:00:00
2	Khilan	1500	2009-10-08 00:00:00

2	Khilan	1560	2009-11-20 00:00:00
2	Khilan	2060	2008-05-20 00:00:00
3	kaushik	3000	2009-10-08 00:00:00
3	kaushik	1500	2009-10-08 00:00:00
3	kaushik	1560	2009-11-20 00:00:00
3	kaushik	2060	2008-05-20 00:00:00
4	Chaitali	3000	2009-10-08 00:00:00
4	Chaitali	1500	2009-10-08 00:00:00
4	Chaitali	1560	2009-11-20 00:00:00
4	Chaitali	2060	2008-05-20 00:00:00
5	Hardik	3000	2009-10-08 00:00:00
5	Hardik	1500	2009-10-08 00:00:00
5	Hardik	1560	2009-11-20 00:00:00
5	Hardik	2060	2008-05-20 00:00:00
6	Komal	3000	2009-10-08 00:00:00
6	Komal	1500	2009-10-08 00:00:00
6	Komal	1560	2009-11-20 00:00:00
6	Komal	2060	2008-05-20 00:00:00
7	Muffy	3000	2009-10-08 00:00:00
7	Muffy	1500	2009-10-08 00:00:00
7	Muffy	1560	2009-11-20 00:00:00
7	Muffy	2060	2008-05-20 00:00:00